

对单细胞数据进行比对和量化

1. **比对**：将测序读段比对到参考基因组。
2. **量化**：计算每个基因在每个细胞中的表达量。
3. **创建 AnnData 对象**：将量化后的数据导入到 `AnnData` 对象中。

wget -O cellranger-8.0.1.tar.gz "https://cf.10xgenomics.com/releases/cell-exp/cellranger-8.0.1.tar.gz?Expires=1731358420&Key-Pair-Id=APKA17S6A5RYOXBWRPDA&Signature=hgd7eY8G9M0lvmd79XYZTbjmfrF-DOWSbMTxh2tJFLvS3FLv267RmhdLK6JnaQbI8iduTFbb6YVvk8FDDeAHgB-edZWvk9JY74s9nAerfNPNU4eMoCzsAktOgo3ZA3pN55gTc6d8TSxqO36XQMxIRGpNLQWaKiZtn~HOIW6m2SeS4~RN7NQ7o2xFivJKGZOO~JEyMnEDkMVMYzb~GIHpcinDdj~D1j9wyaV4GaUXC1E3I2dnTZzj~LYqCW5TD~eNLIIMsgSoCEtBQa-sUml8gVfSKAg2GGLWUEauim9r6daK2f9G7ziiO-qsjEdsdPfz3O5UuyshEH8518DMc4prt-jw__"

```
curl -O "https://cf.10xgenomics.com/supp/cell-exp/refdata-gex-GRCm39-2024-A.tar.gz"
```

```
cellranger count --id=sample_id \ #输出文件夹的名字
--transcriptome=/path/to/reference/transcriptome \ #参考基因组路
径
--fastqs=/path/to/fastq/files \ #测序数据的路径
--sample=sample_name #指定sample的配套文件前缀
```

```
.outs
├── analysis 【数据分析文件夹】
│   ├── clustering 【聚类，图聚类和k-means聚类】
│   ├── diffexp 【差异分析】
│   ├── pca 【主成分分析线性降维】
│   └── tsne 【非线性降维信息】
├── cloupe.cloupe 【Loupe Cell Browser 输入文件】
├── filtered_feature_bc_matrix 【很重要，Seurat, Monocle的输入文件】
│   ├── barcodes.tsv.gz
│   ├── features.tsv.gz
│   └── matrix.mtx.gz
├── filtered_feature_bc_matrix.h5 【过滤掉的barcode信息HDF5 format】
├── metrics_summary.csv 【CSV format数据摘要】
├── molecule_info.h5 【aggregate的时候会用到的文件】
├── raw_feature_bc_matrix 【原始barcode信息】
│   ├── barcodes.tsv.gz
│   ├── features.tsv.gz
│   └── matrix.mtx.gz
├── possorted_genome_bam.bam 【比对文件】
├── possorted_genome_bam.bam.bai 【索引文件】
├── raw_feature_bc_matrix.h5 【原始barcode信息HDF5 format】
├── web_summary.html 【网页简版报告以及可视化】
└── *_gene_bar.csv_temp 【过程文件】
```

###如果有特殊的分析需求或者不同比对参数才需要

```
STAR --runThreadN 16 \
    --genomeDir /path/to/genomeDir \
    --readFilesCommand zcat \
    --readFilesIn /path/to/R1.fastq.gz /path/to/R2.fastq.gz \
    --soloType CB_UMI_Simple \
    --soloBarcodeReadLength 16 \
    --soloUMIreadLength 12 \
    --outSAMtype None \
    --outFileNamePrefix /path/to/output/
```

Scanpy安装:

[Installation — scanpy](#)

```
conda install -c conda-forge scanpy python-igraph leidenalg
conda install conda-forge::scanpy
```

or

```
pip install --user scanpy
```

```
Python深色版本 ▾ | 📄

import scanpy as sc
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 加载示例数据
adata = sc.datasets.pbmc3k()

# 预处理数据
sc.pp.filter_cells(adata, min_genes=200)
sc.pp.filter_genes(adata, min_cells=3)
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)
sc.pp.highly_variable_genes(adata, min_mean=0.0125, max_mean=3, min_disp=0.5)
adata = adata[:, adata.var.highly_variable]
adata.raw = adata

# 降维和聚类
sc.tl.pca(adata, svd_solver='arpack')
sc.pp.neighbors(adata, n_neighbors=10, n_pcs=40)
sc.tl.umap(adata)
sc.tl.leiden(adata)

# 可视化
sc.pl.umap(adata, color=['leiden', 'n_genes_by_counts', 'total_counts'])
sc.pl.tsne(adata, color=['leiden', 'n_genes_by_counts', 'total_counts'])
sc.pl.pca(adata, color='leiden')

# 差异表达分析
sc.tl.rank_genes_groups(adata, 'leiden', method='t-test')
sc.pl.rank_genes_groups(adata, n_genes=25, sharey=False)

# 保存结果
adata.write('processed_data.h5ad')
```

```
import scanpy as sc
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
adata = sc.read_10x_h5('path/to/your/data.h5')
```

```
adata = sc.read_10x_mtx('path/to/your/data_directory')
```

```
# 读取表达矩阵
expression_matrix = pd.read_csv('path/to/expression_matrix.csv', index_col=0)

# 读取细胞注释（可选）
obs = pd.read_csv('path/to/cell_annotations.csv', index_col=0)

# 读取基因注释（可选）
var = pd.read_csv('path/to/gene_annotations.csv', index_col=0)

# 创建 AnnData 对象
adata = sc AnnData(X=expression_matrix.values, obs=obs, var=var)
```

```
adata = sc.read_h5ad('path/to/your/data.h5ad')
```

```
adata = sc.read_loom('path/to/your/data.loom')
```

预处理数据

`sc.pp.filter_cells(adata, min_genes=200)` : 过滤掉表达基因数少于200的细胞

`sc.pp.filter_genes(adata, min_cells=3)` : 过滤掉在少于3个细胞中表达的基因

`sc.pp.normalize_total(adata, target_sum=1e4)` : 将每个细胞的总UMI计数归一化为10,000

`sc.pp.log1p(adata)` : 对数据进行对数变换, 以减少高表达基因的影响

`sc.pp.highly_variable_genes(adata, min_mean=0.0125, max_mean=3, min_disp=0.5)` : 识别高变基因, 这些基因在不同细胞中表达差异较大

`adata = adata[:, adata.var.highly_variable]` : 保留高变基因

`adata.raw = adata` : 将处理前的数据存储在 `adata.raw` 中, 以便后续参考

降维和聚类

`sc.tl.pca(adata, svd_solver='arpack')` : 使用主成分分析 (PCA) 进行降维

`sc.pp.neighbors(adata, n_neighbors=10, n_pcs=40)` : 构建细胞之间的邻接图, 使用前40个主成分

`sc.tl.umap(adata)` : 使用UMAP进行降维, 以便于可视化

`sc.tl.leiden(adata)` : 使用Leiden算法进行细胞聚类

可视化

`sc.pl.umap(adata, color=['leiden', 'A gene symbol'])`

`sc.pl.umap(adata, color=['leiden', 'n_genes_by_counts', 'total_counts'])` : 绘制UMAP图, 颜色表示聚类结果、每个细胞的基因数和总UMI计数

`sc.pl.tsne(adata, color=['leiden', 'n_genes_by_counts', 'total_counts'])` : 绘制t-SNE图, 颜色表示聚类结果、每个细胞的基因数和总UMI计数

`sc.pl.pca(adata, color='leiden')` : 绘制PCA图, 颜色表示聚类结果

`pic("plot/unmap_cluster_feature.pdf")`

`adata.write("data/test.h5ad")`

##差异表达分析

`sc.tl.rank_genes_groups(adata, 'leiden', method='t-test')` : 进行差异表达分析, 找出不同聚类之间的差异基因

`sc.pl.rank_genes_groups(adata, n_genes=25, sharey=False)` : 绘制差异表达基因的排名图, 显示每个聚类中排名前25的差异基因

`sc.tl.rank_genes_groups(adata, 'leiden', group=['0'], reference='1', method='wilcoxon')`

`sc.pl.rank_genes_groups(adata, group=['0'], n_genes=8)`

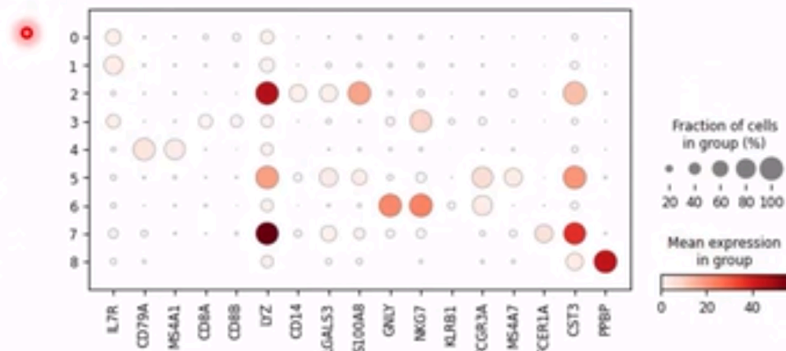
`pic("plot/diff_feature_0v1.pdf")`

`sc.pl.rank_genes_groups_violin(adata, groups='0', n_genes=8)`


```
pic("plot/diff_feature_n_genes.pdf")
```

细胞亚群添加标注

```
marker_genes = ['IL7R', 'CD79A', 'MS4A1', 'CD8A', 'CD8B', 'LYZ', 'CD14', 'LGALS3', 'S100A8',  
'GNLY', 'NKG7', 'KLRB1', 'FCGR3A', 'MS4A7', 'FCER1A', 'CST3', 'PPBP']  
sc.pl.dotplot(adata, marker_genes, groupby='leiden')  
pic("plot/leiden_marker_dotplot.pdf")
```



```
new_cluster_names = ['CD4 T', 'CD4 T', 'CD14 Monocytes', 'B', 'CD8 T', 'FCGR3A Monocytes',  
'NK', 'Dendritic', 'Megakaryocytes']  
new_cluster_names_unique=list(set(new_cluster_names))  
new_cluster_names_unique.sort(key=new_cluster_names.index)  
celltype = [new_cluster_names[int(i)] for i in adata.obs.leiden.values.tolist()]  
adata.obs.leiden = celltype  
adata.obs.leiden = adata.obs.leiden.astype('category')  
adata.obs.leiden.cat.set_categories(new_cluster_names_unique, inplace=True)
```

```
with plt.rc_context({'figure.figsize':(10, 10)}):  
    sc.pl.umap(adata, color='leiden', legend_loc='on data', title="", frameon=True,  
    legend_fontsize='x-small')
```

```
pic("plot/celltype_umap.pdf")  
sc.pl.dotplot(adata, marker_genes, groupby='leiden')  
pic("plot/celltype_marker_dotplot.pdf")  
sc.pl.stacked_violin(adata, marker_genes, groupby='leiden', rotation=90)  
pic("plot/celltype_marker_violin.pdf")
```

AnnData object with $n_obs \times n_vars = 2638 \times 2000$

n_obs: 细胞数目

n_vars: 基因数目

obs: 相当于细胞的meta.data

var: 相当于基因的meta.data

uns: 存储分析方法及参数

obsm: 存储降维坐标等多维信息

obsp: 存储点与点之间的关系

layers: 存储数据，类似于Seurat中的data、counts卡槽

```
>>> ad
AnnDat
  obr
  ounts_
  va
  pout_b
  rsions
  un
  map'
  obr
  va
  obr
```

```
adata.obs.index.to_list() # 获取细胞名
```

```
adata.var.index.to_list() # 获取基因名
```

```
adata.n_obs # 细胞数目
```

```
adata.n_vars # 基因数目
```

```
adata.obs['cluster'].values.tolist()[1:5] # 查看obs某列的值,若没有cluster列名则无法查看
```

```
pd.DataFrame({'barcode':adata.obs.index.to_list(),'celltype':adata.obs.leiden.to_list()}) # 可查看每个细胞barcode对应细胞类型
```

```
adata.obs.leiden.values.categories.tolist() # 可查看细胞类型种类
```

```
adata[:, adata.var.highly_variable].var.index # 获取当前object的Variable feature levels
```

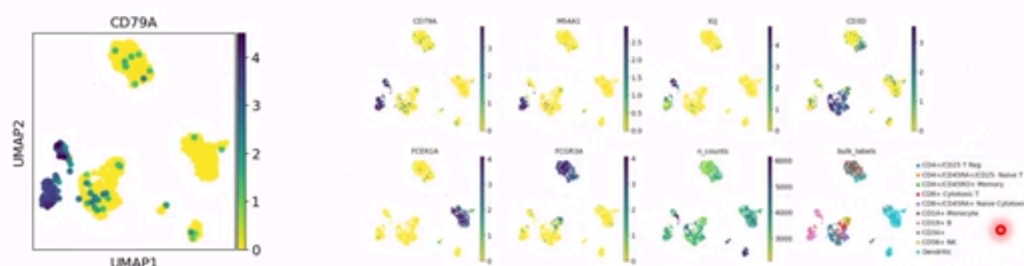
```
adata.obs['cluster']=adata.obs.leiden # 存储旧的细胞标签
```

```
sc.pl.umap(pbm, color='CD79A')
```

```
pic('plot/gene.pdf')
```

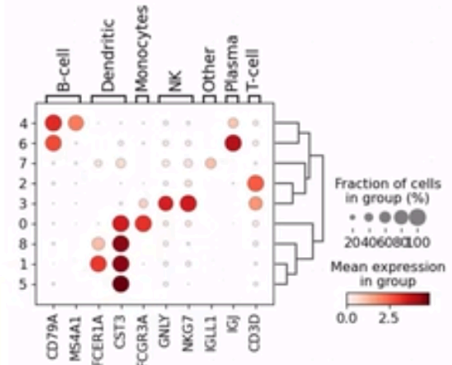
```
sc.pl.umap(pbm, color=['CD79A', 'MS4A1', 'IGJ', 'CD3D', 'FCER1A', 'FCGR3A', 'n_counts', 'bulk_labels'],  
s=50, frameon=False, ncols=4, vmax='p99')
```

```
pic('plot/gene.pdf')
```



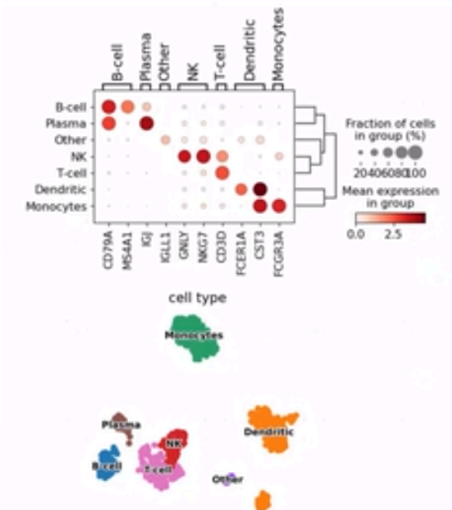
```
marker_genes_dict = {
    'B-cell': ['CD79A', 'MS4A1'],
    'Dendritic': ['FCER1A', 'CST3'],
    'Monocytes': ['FCGR3A'],
    'NK': ['GNLY', 'NKG7'],
    'Other': ['IGLL1'],
    'Plasma': ['IGJ'],
    'T-cell': ['CD3D'],
}

sc.tl.leiden(pbm, key_added='clusters', resolution=0.5)
sc.pl.dotplot(pbm, marker_genes_dict, 'clusters',
dendrogram=True)
pic('plot/gene.pdf')
```



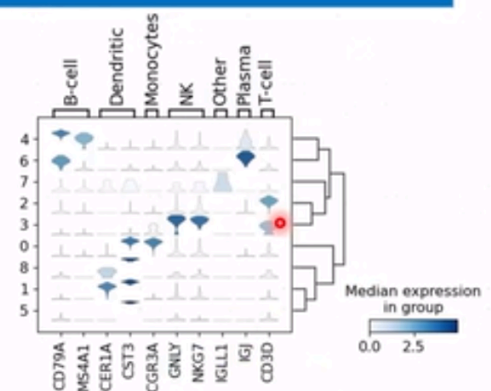
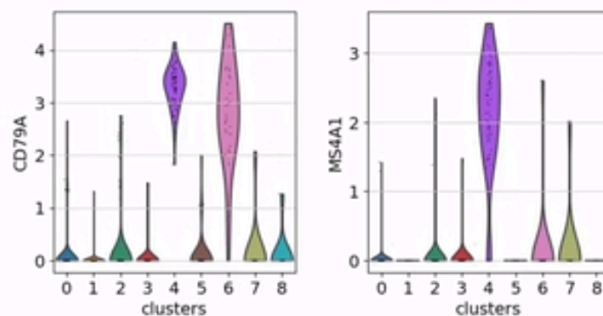
```
cluster2annotation = {
    '0': 'Monocytes',
    '1': 'Dendritic',
    '2': 'T-cell',
    '3': 'NK',
    '4': 'B-cell',
    '5': 'Dendritic',
    '6': 'Plasma',
    '7': 'Other',
    '8': 'Dendritic',
}

pbm.obs['cell type'] =
pbm.obs['clusters'].map(cluster2annotation).astype('category')
sc.pl.dotplot(pbm, marker_genes_dict, 'cell type',
dendrogram=True)
pic('plot/gene.pdf')
sc.pl.umap(pbm, color='cell type', legend_loc='on data',
frameon=False, legend_fontsize=10, legend_fontoutline=2)
pic('plot/gene.pdf')
```



Single cell multi-omics for precision medicine

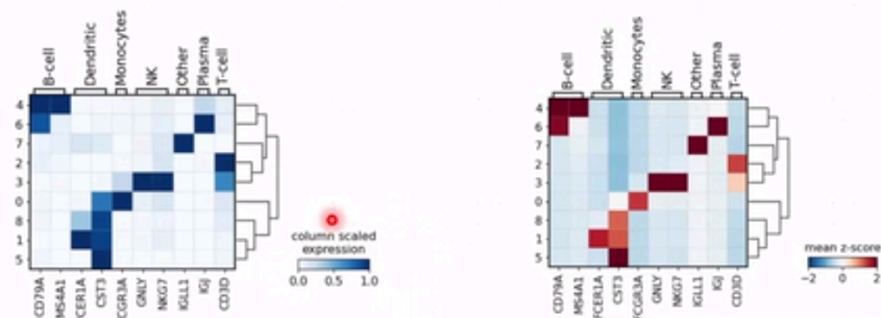
```
sc.pl.violin(pbm, ['CD79A', 'MS4A1'], groupby='clusters')
pic('plot/gene.pdf')
ax = sc.pl.stacked_violin(pbm, marker_genes_dict, groupby='clusters', swap_axes=False,
dendrogram=True)
pic('plot/gene.pdf')
```




```

sc.pl.matrixplot(pbmc, marker_genes_dict, 'clusters', dendrogram=True, cmap='Blues',
standard_scale='var', colorbar_title='column scaled\nexpression')
pic('plot/gene.pdf')
pbmc.layers['scaled'] = sc.pp.scale(pbmc, copy=True).X
sc.pl.matrixplot(pbmc, marker_genes_dict, 'clusters', dendrogram=True, colorbar_title='mean z-
score', layer='scaled', vmin=-2, vmax=2, cmap='RdBu_r')
pic('plot/gene.pdf')

```



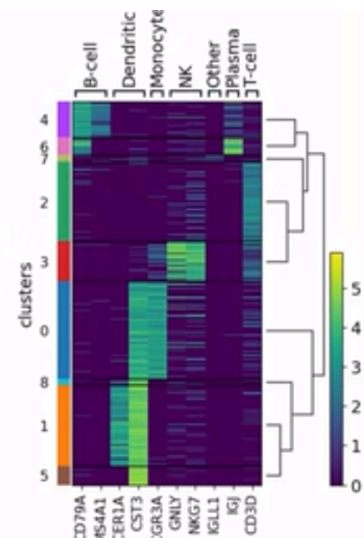
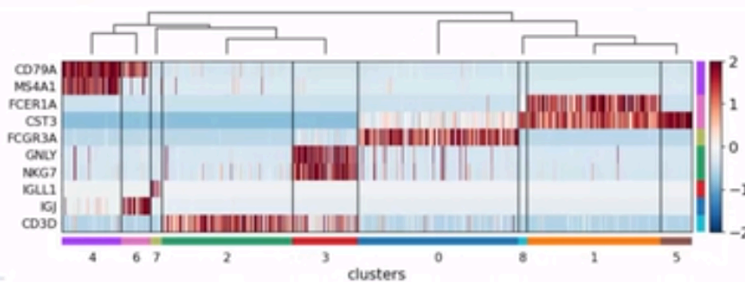
Single cell multi-omics for precision

```

ax = sc.pl.heatmap(pbmc, marker_genes_dict, groupby='clusters',
cmap='viridis', dendrogram=True)
pic('plot/gene.pdf')

ax = sc.pl.heatmap(pbmc, marker_genes_dict, groupby='clusters',
layer='scaled', vmin=-2, vmax=2, cmap='RdBu_r', dendrogram=True,
swap_axes=True, figsize=(11,4))
pic('plot/gene.pdf')

```

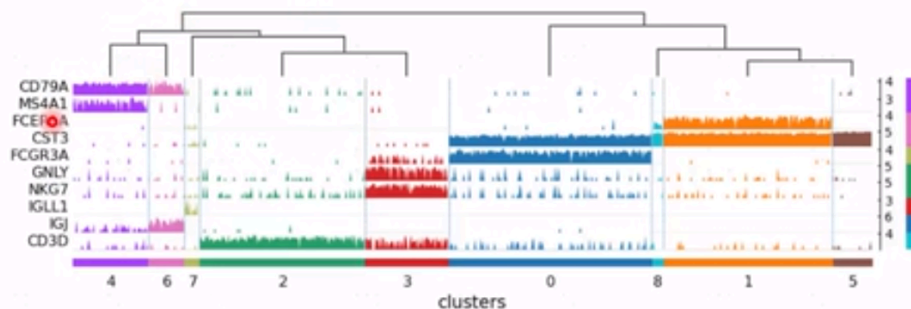


Single cell multi-omics for precision

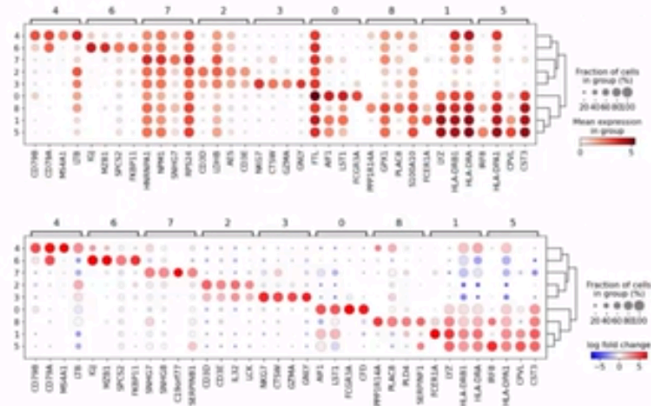
```

ax = sc.pl.tracksplot(pbmc, marker_genes_dict, groupby='clusters', dendrogram=True)
pic('plot/gene.pdf')

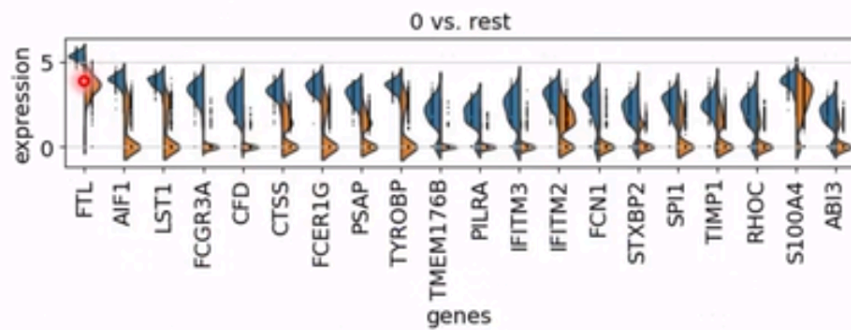
```



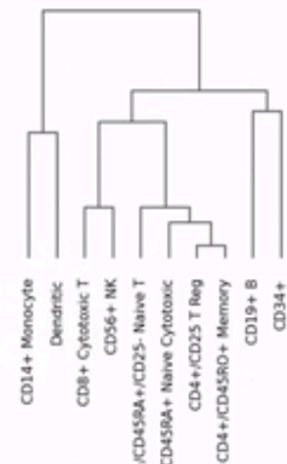
```
sc.tl.rank_genes_groups(pbmc, groupby='clusters', method='wilcoxon')
sc.pl.rank_genes_groups_dotplot(pbmc, n_genes=4)
pic('plot/gene.pdf')
```



```
with rc_context({'figure.figsize': (9, 1.5)}):
    sc.pl.rank_genes_groups_violin(pbmc, n_genes=20, groups=['0'], jitter=False)
pic('plot/gene.pdf')
```



```
sc.tl.dendrogram(pbmc, 'bulk_labels')
ax = sc.pl.dendrogram(pbmc, 'bulk_labels')
pic('plot/gene.pdf')
```



保存结果

```
adata.write('processed_data.h5ad')
```

cellranger 及 scanpy目的讲解

`cellranger count` 和 `scanpy` 的质控和比对步骤实际上是互补的，而不是冲突的。它们在单细胞 RNA 测序数据分析的不同阶段发挥作用：

1. cellranger count:

- **初步处理**：从原始 FASTQ 文件开始，进行细胞条形码和UMI处理、比对到参考基因组、定量基因表达。
- **初步质控**：过滤掉低质量的读数和细胞，生成一个初步的高质量基因表达矩阵。
- **输出标准化数据**：生成标准化的基因表达矩阵和其他相关文件，供后续分析使用。

2. scanpy:

- **进一步质控**：在 `cellranger count` 生成的基因表达矩阵基础上，进行更细致的质控步骤，如去除双重细胞、低质量细胞和批次效应。
- **数据预处理**：归一化、对数转换、去除高变基因等。
- **高级分析**：降维（PCA、t-SNE、UMAP）、聚类分析、差异表达分析等。

互补性

- **初步质控 vs. 进一步质控**：`cellranger count` 进行的是初步质控，确保数据的基本质量。而 `scanpy` 提供了更灵活和细致的质控选项，可以根据具体研究需求进行调整。
- **比对和定量**：`cellranger count` 负责将读数比对到参考基因组并定量基因表达，这一步在 `scanpy` 中不再重复。
- **数据分析**：`scanpy` 提供了丰富的工具用于数据分析和可视化，是在 `cellranger count` 生成的标准化数据基础上进行的。

实践

```
# 设置日志级别
sc.settings.verbosity = 1 # verbosity: errors (0), warnings (1), info (2), hints (3)

# 打印版本信息
sc.logging.print_versions()

# 设置绘图参数
sc.settings.set_figure_params(dpi=150, frameon=False, figsize=(3, 3))
```

```
>>> sc.logging.print_versions()
-----
anndata      0.10.9
scanpy       1.10.3
-----
PIL          11.0.0
colorama     0.4.6
cyclor       0.12.1
cython_runtime NA
dateutil     2.9.0
h5py         3.12.1
igraph       0.11.6
joblib       1.4.2
kiwisolver   1.4.7
legacy_api_wrap NA
leidenalg    0.10.2
llvmlite     0.43.0
matplotlib   3.9.2
mpl_toolkits NA
natsort      8.4.0
numba        0.60.0
numpy        2.0.2
packaging    24.1
pandas       2.2.3
pyparsing    3.2.0
pytz         2024.1
scipy        1.14.1
session_info 1.0.0
six          1.16.0
sklearn      1.5.2
texttable    1.7.0
threadpoolctl 3.5.0
-----
Python 3.12.7 | packaged by conda-forge | (main, Oct  4 2024, 16:05:46) [GCC 13.3.0]
Linux-6.8.0-40-generic-x86_64-with-glibc2.35
```

```
import scanpy as sc
import anndata as ad
import scrublet as scr
import scanpy.external as sce
import pandas as pd
import numpy as np
from pandas import Categorical
from sklearn.linear_model import LinearRegression
from scipy.sparse import issparse
import matplotlib.pyplot as plt
```

```
# 确保 `sample` 列有相同的类别
adata1 = sc.read_10x_mtx('/home/chen/jiangjing/10.singlecell/scanpy/CD45-
1_filtered_feature_bc_matrix/')
adata2 =
sc.read_10x_mtx('/home/chen/jiangjing/10.singlecell/scanpy/IFNR_filtered_feature_
bc_matrix/')
adata1.obs['sample'] = 'CD45-1'
adata2.obs['sample'] = 'IFNR'
adata1.obs_names_make_unique()
adata2.obs_names_make_unique()
adata1.obs['sample'] = Categorical(adata1.obs['sample'], categories=['CD45-1',
'IFNR'])
adata2.obs['sample'] = Categorical(adata2.obs['sample'], categories=['CD45-1',
'IFNR'])
```

```
# 然后合并
adata = ad.concat([adata1, adata2])
adata.obs_names_make_unique()
adata = adata.copy()
```

```
#人是MT, 鼠是mt
# 标识线粒体基因
adata.var["mt"] = adata.var_names.str.startswith("mt-")

# 标识核糖体基因
adata.var["ribo"] = adata.var_names.str.startswith(("RPS", "RPL"))

# 标识血红蛋白基因
adata.var["hb"] = adata.var_names.str.contains("^HB[^(P)]")

# 计算 QC 指标
sc.pp.calculate_qc_metrics(
    adata,
    qc_vars=["mt", "ribo", "hb"],
    inplace=True,
    log1p=True
)

# 查看结果
print(adata.obs.head())
print(adata.var.head())
sc.pl.violin(
    adata,
    ["n_genes_by_counts", "total_counts", "pct_counts_mt"],
    jitter=0.4,
    multi_panel=True,
)
sc.pl.scatter(adata, "total_counts", "n_genes_by_counts", color="pct_counts_mt")
```

```
>>> print(adata.obs.head())
              sample
AAACCCAAGACCTTG-1  CD45-1
AAACCCAAGAGTTGAT-1  CD45-1
AAACCCAAGATAGGGA-1  CD45-1
AAACCCAAGCCGTTGC-1  CD45-1
AAACCCAAGGGAGTGG-1  CD45-1
>>> print(adata.var.head())
              mt    ribo    hb
Xkr4         False  False  False
Gm1992        False  False  False
Gm19938       False  False  False
Gm37381       False  False  False
Rp1          False  False  False
```



```
>>> print(adata.var.head())
      mt      ribo      hb  n_cells_by_counts  mean_counts  logip_mean_counts  pct_dropout_by_counts  total_counts  logip_total_counts
Xkr4  False  False  False           54      0.002839      0.002835      99.771196           67.0      4.219588
Gm1992  False  False  False           1      0.000042      0.000042      99.995763           1.0      0.693147
Gm1993B  False  False  False           2      0.000085      0.000085      99.991526           2.0      1.098612
Gm37381  False  False  False           0      0.000000      0.000000      100.000000           0.0      0.000000
Rp1  False  False  False           91      0.005466      0.005451      99.614423          129.0      4.867535
>>> print(adata.obs.head())
      sample  n_genes_by_counts  logip_n_genes_by_counts  total_counts  logip_total_counts  ...  logip_total_counts_ribo  pct_counts_ribo  total_counts_hb  logip_total_counts_hb  pct_counts_hb
AAACCCAAGACCTTTG-1  CD45-1      2691      7.645876      5088.0      8.518991  ...      0.0      0.0      0.0      0.0      0.0
AAACCCAAGAGTGTGAT-1  CD45-1      3379      8.125631      16883.0      9.287672  ...      0.0      0.0      0.0      0.0      0.0
AAACCCAAGATAGGG-1  CD45-1      1940      7.575972      4889.0      8.494947  ...      0.0      0.0      0.0      0.0      0.0
AAACCCAAGCGCTTGC-1  CD45-1      2171      7.683484      5111.0      8.539346  ...      0.0      0.0      0.0      0.0      0.0
AAACCCAAGGGAGTGG-1  CD45-1      4523      8.417152      18423.0      9.821409  ...      0.0      0.0      0.0      0.0      0.0
```

adata.obs 列含义

sample:

表示每个细胞所属的样本编号或标签。

n_genes_by_counts:

每个细胞中非零表达的基因数。

logip_n_genes_by_counts:

n_genes_by_counts 的对数变换加1 (**log1p**)，用于减少数据的偏斜性。

total_counts:

每个细胞中的总UMI (**Unique Molecular Identifiers**) 计数，即所有基因的总表达量。

logip_total_counts:

total_counts 的对数变换加1 (**log1p**)，用于减少数据的偏斜性。

pct_counts_mt:

每个细胞中线粒体基因的UMI计数占总UMI计数的百分比。

total_counts_ribo:

每个细胞中核糖体基因的UMI计数。

logip_total_counts_ribo:

total_counts_ribo 的对数变换加1 (**log1p**)。

pct_counts_ribo:

每个细胞中核糖体基因的UMI计数占总UMI计数的百分比。

total_counts_hb:

每个细胞中血红蛋白基因的UMI计数。

logip_total_counts_hb:

total_counts_hb 的对数变换加1 (**log1p**)。

pct_counts_hb:

每个细胞中血红蛋白基因的UMI计数占总UMI计数的百分比。

adata.var 列含义

mt:

布尔值，表示该基因是否为线粒体基因。

ribo:

布尔值，表示该基因是否为核糖体基因。

hb:

布尔值，表示该基因是否为血红蛋白基因。

n_cells_by_counts:

表达该基因的细胞数。

mean_counts:

该基因在所有细胞中的平均UMI计数。

log1p_mean_counts:

mean_counts 的对数变换加1 (**log1p**)，用于减少数据的偏斜性。

pct_dropout_by_counts:

该基因在所有细胞中未表达的百分比 (**dropout rate**)。

total_counts:

该基因在所有细胞中的总UMI计数。

log1p_total_counts:

total_counts 的对数变换加1 (**log1p**)，用于减少数据的偏斜性。

```
>>> adata
AnnData object with n_obs × n_vars = 23601 × 33696
```

#过滤掉总基因数大于6000的细胞

```
adata = adata[adata.obs['n_genes_by_counts'] < 6000, :]
```

#过滤掉线粒体基因比例超过20%的细胞

```
adata = adata[adata.obs['pct_counts_mt'] < 20, :]
```

#检查过滤结果

```
print(f"Number of cells after filtering: {adata.n_obs}")
```

```
print(f"Number of genes after filtering: {adata.n_vars}")
```

```
>>> print(f"Number of cells after filtering: {adata.n_obs}")
Number of cells after filtering: 23580
>>>
>>> print(f"Number of genes after filtering: {adata.n_vars}")
Number of genes after filtering: 33696
```

```
#adata.obs["n_genes"] = adata.obs["n_genes_by_counts"]
sc.pp.filter_genes(adata, min_cells=3)
sc.pp.filter_cells(adata, min_genes=200)
```

#检测双细胞[GitHub - swolock/scrublet: Detect doublets in single-cell RNA-seq data](https://github.com/swolock/scrublet)

```
# 使用 scrublet 检测双细胞
scrub = scr.Scrublet(adata.X)
doublet_scores, predicted_doublets = scrub.scrub_doublets()

# 将双细胞检测结果添加到 adata.obs 中
adata.obs['doublet_score'] = doublet_scores
adata.obs['predicted_doublet'] = predicted_doublets

# 绘制双细胞得分的分布图
```

```
plt.hist(doublet_scores, bins=50, alpha=0.5, color='blue')
plt.xlabel('Doublet Score')
plt.ylabel('Number of Cells')
plt.title('Distribution of Doublet Scores')
plt.show()

# 手动指定双细胞得分阈值
threshold = 0.1 # 你可以根据分布图调整这个值

# 调用 call_doublets 方法并指定阈值
scrub.call_doublets(threshold=threshold)
adata.obs['predicted_doublet_manual'] = scrub.predicted_doublets_

# 过滤掉预测为双细胞的细胞
adata = adata[~adata.obs['predicted_doublet_manual'], :]
```

#优先选择表达Ptprc的基因过滤免疫细胞

```
>>> ptprc_expression = adata[:, 'PTPRC'].X.toarray().flatten()
Traceback (most recent call last):
  File "/home/chen/anaconda3/envs/singlecell/lib/python3.12/site-packages/pandas/core/indexes/base.py", line 3805, in get_loc
    return self._engine.get_loc(casted_key)
  File "pandas/_libs/index.pyx", line 167, in pandas._libs.index.IndexEngine.get_loc
  File "pandas/_libs/index.pyx", line 196, in pandas._libs.index.IndexEngine.get_loc
  File "pandas/_libs/hashtable_class_helper.pxi", line 7081, in pandas._libs.hashtable.PyObjectHashTable.get_item
  File "pandas/_libs/hashtable_class_helper.pxi", line 7089, in pandas._libs.hashtable.PyObjectHashTable.get_item
KeyError: 'PTPRC'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/chen/anaconda3/envs/singlecell/lib/python3.12/site-packages/anndata/_core/anndata.py", line 1021, in __getitem__
    oidx, vidx = self._normalize_indices(index)
  File "/home/chen/anaconda3/envs/singlecell/lib/python3.12/site-packages/anndata/_core/anndata.py", line 1002, in _normalize_indices
    return _normalize_indices(index, self.obs_names, self.var_names)
  File "/home/chen/anaconda3/envs/singlecell/lib/python3.12/site-packages/anndata/_core/index.py", line 39, in _normalize_indices
    ax1 = _normalize_index(ax1, names1)
  File "/home/chen/anaconda3/envs/singlecell/lib/python3.12/site-packages/anndata/_core/index.py", line 76, in _normalize_index
    return index.get_loc(indexer) # int
  File "/home/chen/anaconda3/envs/singlecell/lib/python3.12/site-packages/pandas/core/indexes/base.py", line 3812, in get_loc
    raise KeyError(key) from err
KeyError: 'PTPRC'
>>> print('PTPRC' in adata.var_names)
False
>>> import re
>>> matching_genes = [gene for gene in adata.var_names if re.search(r'ptprc', gene, re.IGNORECASE)]
>>> print(matching_genes)
['Ptprc', 'Ptprcp']
```

```
import re
if 'PTPRC' not in adata.var_names:
    print("Gene 'PTPRC' not found. Checking for similar gene names...")
    import re
    matching_genes = [gene for gene in adata.var_names if re.search(r'ptprc',
gene, re.IGNORECASE)]
    if matching_genes:
        print(f"Found matching genes: {matching_genes}")
        gene_name = matching_genes[0] # 选择第一个匹配的基因
    else:
        raise ValueError("No matching gene found.")
else:
    gene_name = 'PTPRC'

# 获取 PTPRC 基因的表达量
ptprc_expression = adata[:, gene_name].X.toarray().flatten()

# 设置一个阈值，例如选择表达量大于某个值的细胞
```

```

threshold = 1.0 # 你可以根据实际情况调整这个值

# 创建一个布尔数组，标记表达量大于阈值的细胞
high_ptprc_cells = ptprc_expression > threshold

# 过滤出优先表达 PTPRC 的细胞
adata_high_ptprc = adata[high_ptprc_cells, :]

# 绘制 PTPRC 表达量的直方图
plt.hist(ptprc_expression, bins=50, alpha=0.5, color='blue')
plt.axvline(threshold, color='red', linestyle='dashed', linewidth=2)
plt.xlabel(f'{gene_name} Expression')
plt.ylabel('Number of Cells')
plt.title(f'Distribution of {gene_name} Expression')
plt.show()

# 查看过滤后的细胞数量
print(f'Number of cells expressing {gene_name} above the threshold:
{adata_high_ptprc.n_obs}')

# 保存结果
adata_high_ptprc.write(f'filtered_adata_high_{gene_name}.h5ad')

```

```

>>> print(f'Number of cells expressing Ptprc above the threshold: {adata_high_ptprc.n_obs}')
Number of cells expressing Ptprc above the threshold: 17310

```

```

adata.layers["counts"] = adata.X.copy()
# Normalizing to median total counts
sc.pp.normalize_total(adata, target_sum=10000)
# Logarithmize the data
sc.pp.log1p(adata)
# 检查数据是否已经对数变换
if not (adata.X.min() >= 0).all():
    print("Data is already log-transformed. Skipping log1p transformation.")
else:
    sc.pp.log1p(adata)

# 如果需要回溯到原始数据，可以使用 adata.layers["counts"]
print("Original counts data stored in adata.layers['counts']:")
print(adata.layers["counts"])
adata.write('normalized_and_log_transformed_data.h5ad')

```

```

#注释高可变基因
sc.pp.highly_variable_genes(adata, n_top_genes=2000, batch_key="sample")
sc.pl.highly_variable_genes(adata)

```

```

#降维
sc.tl.pca(adata)
#获取方差解释比
variance_ratio = adata.uns['pca']['variance_ratio']

for i, ratio in enumerate(variance_ratio[:50]):
    print(f"PC{i+1}: {ratio:.4f}")
# 创建一个 DataFrame

```

```

df = pd.DataFrame({
    'PC': range(1, len(variance_ratio) + 1),
    'Variance Ratio': variance_ratio
})

# 保存为 CSV 文件
df.to_csv('pca_variance_ratio.csv', index=False)
df.to_excel('pca_variance_ratio.xlsx', index=False)

sc.pl.pca_variance_ratio(adata, n_pcs=50, log=True)

sc.pl.pca(
    adata,
    color=["sample", "sample", "pct_counts_mt", "pct_counts_mt"],
    dimensions=[(0, 1), (2, 3), (0, 1), (2, 3)],
    ncols=2,
    size=2,
)

```

```

# 计算总 UMI 计数
if issparse(adata.X):
    adata.obs['total_counts'] = adata.X.sum(axis=1).A1
else:
    adata.obs['total_counts'] = adata.X.sum(axis=1)

# 验证结果
print(adata.obs['total_counts'])

```

```

import bbknn
# 运行 BBKNN
bbknn.bbknn(adata, batch_key='sample')

import scanpy.external as sce
# 运行 Harmony
sce.pp.harmony_integrate(adata, key='sample')

import scanorama
# 运行 Scanorama
adata_integrated = scanorama.correct_scanpy(adata, batch_key='sample')

```

```

sc.pl.umap(adata_high_ptprc, color="sample")

```